

COP2034C Module 6 Programming Assignment

Instructions

In this assignment, we will continue building the Palmer Penguins program from Module 5. In that module, we added loops so the program could display species statistics without repeated output statements and allow the user to explore multiple species during the same program run. We will reorganize selected parts of the program into user-defined functions. This process is called **refactoring**: improving the internal structure of existing code without changing what the program is intended to do.

Functions allow us to give a name to a group of related statements and then call that function when the operation is needed. Functions can make a program easier to read, reduce repeated code, and separate a larger program into smaller pieces that can be developed and tested individually.

We will practice:

- defining and calling functions,
- passing values to functions using parameters and arguments,
- returning values from functions,
- creating a function that produces output without returning a value,
- using branches inside functions,
- using a loop inside a function,
- and organizing a program into smaller, reusable operations.

The completed Module 6 program should produce essentially the same results as Module 5. The primary change is how the program is organized internally. As before, work incrementally: create or modify one function, run the program, verify that it works correctly, and then continue.

1. Start with Your Module 5 Solution

Open the Palmer Penguins notebook you completed for Module 5. Copy the notebook for Module 6. A suggested name is:

PalmerPenguinsM6.ipynb

The Python file you eventually download should have a meaningful name such as:

PalmerPenguinsM6.py

Do not rewrite the program from the beginning. Module 6 should build on your completed Module 5 program.

2. Update the ID Header

Update the ID header at the top of the program:

```
# PalmerPenguinsM6.py
# Your Name
# Submission Date
# Use functions to organize Palmer Penguin analysis
```

Leave a blank line after the header.

3. Retain Your Module 5 Code

Keep the constants, lists, calculations, loops, branches, and interactive species exploration from Module 5. Continue using the three parallel lists:

```
species_names
species_counts
species_percentages
```

The same index should continue to identify related information in all three lists.

Some existing Module 5 statements in this assignment will be moved into functions or replaced with function calls. Unless the instructions specifically direct you to replace some code, the existing program behavior should remain.

4. Define Functions Before the Main Program

Place the new function definitions **after the import statement and constants**, but **before the portion of the program that performs the main calculations and output**. A function must be defined before the program can call it.

Use lowercase names with underscores for your functions, following the Python naming conventions demonstrated in this course.

5. Create a Function to Calculate a Percentage

Module 5 calculated the three species percentages using three similar expressions:

```
chinstrap_percent = species_counts[0] / total_penguins * 100
gentoo_percent = species_counts[1] / total_penguins * 100
adelie_percent = species_counts[2] / total_penguins * 100
```

Create a function which performs this calculation named

```
calculate_percentage()
```

The function should have two parameters:

```
count
total
```

The function should calculate

```
count / total * 100
```

and return the resulting calculated percentage. The function definition should begin with

```
def calculate_percentage(count, total):
```

This is known as the function **header**. After creating the function, replace the three existing percentage calculations with three calls to `calculate_percentage()`. For example:

```
chinstrap_percent = \  
    calculate_percentage(species_counts[0], total_penguins)
```

Note the use of the backslash (\) character (prefixed with a space character) and indented follow-on line to continue the statement on the next line. This allows you to keep your statements at less than 80 columns for readability.

Create similar calls for Gentoo and Adelie. The calculated percentages should remain unchanged.

6. Create a Function to Classify a Species

During species exploration, Module 5 used an if-elif-else statement to determine whether a species group was small, medium, or large. Now we will create a function named:

```
classify_species()
```

The function should have one parameter:

```
percent
```

Move the existing percentage-classification logic into this function. The function should return:

- "small" when the percentage is less than 25,
- "medium" when the percentage is at least 25 but less than 40,
- "large" otherwise.

NOTE: the function should return the category, do not print it inside the function.

Inside the interactive portion of the program, replace the existing classification branch with a call such as:

```
size_category = classify_species(selected_percent)
```

The output produced later in the program should continue to use `size_category` exactly as it did in Module 5.

7. Create a Function to Determine Relative Position

Module 5 determined whether the selected species was the largest, smallest, or middle-sized species group. Now we will create a function named

```
get_position_description()
```

Give the function these three parameters:

```
selected_species  
largest_species  
smallest_species
```

Inside the function, use an if-elif-else statement.

```
If selected_species equals largest_species, return  
    largest  
Else If selected_species equals smallest_species, return  
    smallest  
Otherwise, return  
    middle-sized
```

Replace the existing Module 5 branch that creates position_description with a call to the new function:

```
position_description = \  
    get_position_description(selected_species, \  
        largest_species, smallest_species)
```

This demonstrates that a function can contain branches and return different values depending on its arguments. Note the use of the backslash to keep the line length less than 80 columns.

8. Create a Function to Display the Species Statistics

Module 5 used a for loop and enumerate() to display

```
Chinstrap: 68 (19.88%)  
Gentoo: 123 (35.96%)  
Adelie: 151 (44.15%)
```

Move this loop into a function named

```
display_species_statistics()
```

The function should have three parameters:

```
names  
counts  
percentages
```

Inside the function, use the same for loop and enumerate() logic from Module 5.
For example, the function header should be

```
def display_species_statistics(names, counts, percentages):
```

The function should **print** the species statistics. **It should not return a value.** Note that this is different from the classify_species() function described earlier; sometimes functions print output and do not return anything. For these functions we do not assign their calls to a variable, we simply call their name.

In the main portion of the program, replace the original loop with one function call:

```
display_species_statistics(species_names, \
    species_counts, species_percentages)
```

This demonstrates a function that performs output rather than returning a calculated result.

9. Keep the Interactive Loop

Keep the Module 5 sentinel-controlled while loop. The user should still be able to repeatedly enter

```
Chinstrap
Gentoo
Adelie
Done
```

Continue validating the user's selection using

```
if species_choice not in species_names:
```

and continue counting valid selections using

```
valid_selections
```

The important change in this module is that some of the work performed inside the loop will now be handled by function calls. For a valid species, the program should now use

```
classify_species()
and:
get_position_description()
```

instead of containing all of that logic directly inside the main loop.

10. Organize the Main Program in a main() Function

So far, the executable statements in our Palmer Penguins program have appeared directly below the function definitions. We will now organize those statements inside a function named **main()**. The **main()** function represents the starting point for the primary work performed by the program (similar to its use in other programming languages). Using a **main()** function helps separate the program's overall sequence of operations from the smaller helper functions that perform individual tasks. After your other function definitions, define:

```
def main():
```

Move the statements that make up the main program into this function and indent them four spaces so they belong to **main()**. For example, the beginning of the function will look like this:

```
def main():
    # lists containing related penguin data
    species_names = [ SP_CHINSTRAP, SP_GENTOO, SP_ADELIE ]
    species_counts = [ NUM_CHINSTRAP, NUM_GENTOO, NUM_ADELIE ]
    # calculate dataset statistics
    total_species = len(species_names)
    total_penguins = sum(species_counts)
```

Continue moving the remaining executable statements into **main()**, including the calculations, output, interactive while loop, and final exploration summary.

Do not move the following into main():

- the import math statement
- the species and count constants
- the function definitions created earlier in this assignment

After the **main()** function definition is complete, add a call to the function at the bottom of the program (do not indent):

```
main()
```

Python first reads the function definitions and then reaches this call, which begins execution of the main portion of the program.

11. Keep the Remaining Module 5 Logic

We are not moving every statement into a function. Leave the following logic in the main portion of the program:

- determining `selected_index`,

- retrieving the selected species information from the parallel lists,
- comparing the selected count to the average,
- updating valid_selections,
- displaying the selected species details,
- prompting for the next species,
- and displaying the final exploration summary.

A useful guideline for deciding which parts of a program to refactor is to look for code that performs a distinct task, is repeated, or would be easier to understand by relocating it under a descriptive function name. Not every small group of statements needs to become a function; the goal is to improve organization and readability without making the program unnecessarily fragmented.

Expected Program Organization

Your completed program should now have a general organization which looks like this:

```
ID header
import statements
constants
function definitions
    calculate_percentage()
    classify_species()
    get_position_description()
    display_species_statistics()
main() function
    create lists
    calculate dataset statistics using function calls
    display dataset statistics
    explore species using the while loop
    display final summary
main() # call the main function to start the program
```

Expected Output

The expected output is essentially the same as Module 5. The dataset statistics should still include

```
Chinstrap: 68 (19.88%)
Gentoo: 123 (35.96%)
Adelie: 151 (44.15%)
```

A sample interactive run might still look like this:

```
Explore a species:
Enter Chinstrap, Gentoo, Adelie, or Done: Gentoo
```

```
Gentoo has 123 observations, representing 35.96% of the dataset.
```

Gentoo is classified as a medium species group.
Gentoo is the middle-sized species group.
Its observation count is above the dataset average of 114.0.

Enter Chinstrap, Gentoo, Adelie, or Done: Chinstrap

Chinstrap has 68 observations, representing 19.88% of the dataset.
Chinstrap is classified as a small species group.
Chinstrap is the smallest species group.
Its observation count is below the dataset average of 114.0.

Enter Chinstrap, Gentoo, Adelie, or Done: Done

You explored 2 valid species.
Exploration complete.

The assignment changes the **organization of the code**, not the required results. Minor whitespace differences are acceptable as long as the output is readable and contains all required information.

Test Your Functions

Before submitting the assignment, test this sequence during one program run

Chinstrap
Gentoo
Adelie
Emperor
Done

Verify that:

- all three valid species display the same statistics as in Module 5,
- each species receives the correct small, medium, or large classification,
- each species is correctly identified as smallest, middle-sized, or largest,
- Emperor is rejected as an invalid species,
- Done ends the loop,
- and the program reports 3 valid species selections.

This sequence helps exercise each branch of the new functions.

Data Science Connection

Functions are especially useful in data analysis because the same calculations and transformations are often applied repeatedly to different values or different parts of a dataset.

In this assignment, `calculate_percentage()` represents a reusable calculation, while `classify_species()` converts a numeric result into a meaningful category. The same functions could later be applied to many more species without rewriting the calculation or classification logic.

Separating calculations, classifications, and output into functions also makes a data-analysis program easier to test and maintain. When we begin working with larger datasets, a function can perform one well-defined step of the analysis while the main program coordinates those steps.

Readability

Continue following the readability expectations from previous modules: meaningful names, consistent four-space indentation, reasonable line lengths, logical blank lines, and comments identifying major sections. Useful new comments might include

```
# functions for penguin analysis

# calculate dataset statistics

# display species statistics

# repeatedly prompt the user to explore species

# display the exploration summary
```

Function names should clearly describe the operation being performed. Avoid duplicating code that has been moved into a function. Once a calculation or branch has been replaced by a function call, remove the original version from the main program.

Screenshot and Submission

Follow the same submission process used for Module 5. Run the completed program in Google Colab. For the screenshot, show enough of one execution to demonstrate that the program successfully processes multiple species and reaches the final summary. Suggested filenames:

PalmerPenguinsM6.py
PalmerPenguinsM6.png

Upload both files to your forked GitHub repository and submit the repository URL to Canvas. Do not submit the .ipynb notebook.